TriX

Design and Implementation Standard

Version 0.2 | 22 October 2026
Proposed normative baseline

TriX is a language and a set of execution contracts for organising projects, tasks and distributed resources. She is designed to combine accessible expression, strong validation and integration with existing engines. Her initial reference installation uses a Mac cluster, an NVIDIA cluster and heterogeneous terminals; her architecture admits other configurations.

This consolidated edition governs language design, runtime behaviour, extension interfaces, platform qualification and implementation evidence. It is accompanied by an English implementation pack containing contract schemas, examples, requirement and test registers, decision templates and validation tools.

The name, feminine identity, English documentation policy and vendor openness reflect the project direction. Technical clauses are a proposed baseline for adoption. This edition specifies work to be implemented; it does not certify a compiler, runtime, platform or accelerator. The companion validation report measures the documentation package only.

Using this standard

Adopt a revision and select the applicable profiles. Link every mandatory requirement to implementation and validation evidence. Chapter 19 sets out delivery gates and decisions that remain open.

The companion pack provides implementation guidance, structured contracts, examples and registers. Its examples are illustrative; schema checks do not certify a TriX implementation.

Contents: 228 identified requirements, 67 acceptance cases, 22 chapters and 30 technical references.

Contents

1 Identity and purpose 3

2 Normative governance 3

3 Scope and design objectives 4

4 Logical and physical architecture 5

5 Core language contracts 6

6 Concurrency and interoperability 7

7 Authority, security and audit 7

8 Task contracts and failure handling 8

9 Data, models and memory 9

10 Resources, priorities and placement 10

11 Platform identity and compatibility 11

12 Accelerators, managers and remote providers 12

13 Versions, distribution and updates 13

14 Developer tools and user experience 14

15 Language and compiler specification 15

16 Runtime and operational specification 20

17 Qualification, evaluations and evidence 25

18 Contract schemas and public interfaces 30

19 Implementation programme and decisions 33

20 Acceptance catalogue 35

21 Glossary 42

22 Technical sources 43

Contents entries and bracketed source references link to their destinations in the editable edition.

1 Identity and purpose

The spelling TriX plays on the sound of the English word tricks and the idea of transforming a clear description of work into suitable execution. Her feminine identity and the name's appeal are part of the project's creative intent.

IDN-001 Identity. The language MUST be named TriX, preserving capital T and X. Pronouns referring to TriX MUST use the feminine forms she and her in English. This convention does not change the grammatical reference to a compiler, runtime component or artifact. Technical identifiers MAY use the normalisation required by their format.

IDN-002 Purpose. TriX MUST support describing, validating, planning, executing and observing distributed work, including AI models, tools and data processing. She MUST make authority, data location, compatibility and resource conditions explicit.

IDN-003 Independence. The language, central protocol and project data MUST NOT require one exclusive hardware vendor, operating system, model provider or cluster manager. A specialised extension MAY depend on a vendor if that dependency is declared.

IDN-004 Project continuity. Infrastructure changes MUST preserve programs and projects whose semantics and requirements remain satisfied. When a new installation cannot meet a requirement, TriX MUST explain the incompatibility before initiating the affected task's effects.

IDN-005 Documentation language. All authoritative project documentation, specifications, schemas, examples, comments intended for publication, diagnostics catalogues, decision records and implementer instructions MUST be maintained in English. Project documentation in another language requires an explicit later change to this instruction. User interfaces MAY be localised; their authoritative specifications and translation instructions remain in English.

2 Normative governance

Uppercase keywords have the meanings below. Explanations, examples and external references are informative unless a requirement explicitly incorporates them. This document describes project conformance, not regulatory certification.

Keyword	Meaning
MUST	Mandatory within the adopted revision and applicable profile
MUST NOT	Prohibited within that scope
SHOULD or RECOMMENDED	Recommendation; an exception requires a recorded rationale
MAY	Optional capability subject to applicable contracts
Pending	Decision not yet fixed by this revision
Illustrative	Example that does not establish a final grammar or prove implementation

NOR-001 Adopted revision. Each implementation MUST declare its specification revision and conformance profiles. The project authority MUST record adoption and subsequent changes. The existence of this document alone does not constitute formal adoption.

NOR-002 Traceability. Requirement identifiers MUST remain stable. Deliveries MUST connect applicable requirements to implementation, tests and limitations. Retired requirements MUST be marked superseded or withdrawn; their identifiers MUST NOT be reused.

NOR-003 Decisions. Changes affecting semantics, authority, compatibility or guarantees MUST have an architecture decision record, or ADR, covering motivation, alternatives, impact, migration and validation evidence.

NOR-004 Profiles. Conformance MUST distinguish the language core, coordination, local execution, distributed execution, remote access and control clients. Each profile MUST publish mandatory requirements, optional capabilities and profile dependencies before any conformance claim. An implementation MUST NOT exclude requirements needed by functionality it exposes.

NOR-005 Exceptions. Exceptions to recommendations MUST have recorded reasons. An implementation violating a mandatory requirement MUST NOT claim complete conformance for the affected profile. Pending decisions MUST be resolved before the associated capability becomes stable.

NOR-006 Document precedence. For revision 0.2, the numbered English requirements and explicit normative tables in this standard govern behaviour. Companion JSON Schemas govern the structural constraints they express. Examples, templates and generated registers do not override either. A contradiction MUST block the affected conformance claim until a versioned correction resolves it; an implementer MUST NOT silently choose the easier interpretation.

NOR-007 Evidence status. Every capability, decision and test record MUST distinguish proposed, implemented and verified status. A successful document-package check MUST NOT be reported as a TriX runtime, compiler, security or hardware test. Claims MUST identify what was actually executed and what remains untested.

3 Scope and design objectives

The initial product is a language for typed workflows and authorised distributed operations, with a pure expression subset. The first executable form is a versioned task plan generated by an SDK or controlled builder. A full surface language follows validated semantics.

Rust provides a reference for ownership and resource lifetimes; Python for accessible high-level programming and extension ecosystems; Swift for structured concurrency, actors and safe transfer between concurrency domains. These are design precedents, not a promise to merge their implementations or inherit every guarantee. [F01] [F02] [F03]

ESC-001 Initial subset. The first implementation MUST support task contracts, dependencies, policies and execution through adapters. The project SHOULD validate this subset through an SDK, verifiable plans and executors before investing in a general-purpose compiler.

ESC-002 Separation of responsibilities. The specification MUST separate language semantics, plan representation, runtime, adapters and developer tools. A graphical-interface decision MUST NOT change a program's meaning.

ESC-003 Reference implementation. A Rust coordination and execution core, isolated Python integration adapters and Swift clients on Apple platforms are RECOMMENDED. This composition MUST remain an implementation choice; extensions MUST NOT be required to use these languages solely to participate.

ESC-004 Reuse. Existing engines, compilers and distribution mechanisms SHOULD be reused where they satisfy the contracts. A new low-level backend or custom kernel MUST justify its benefit and maintenance cost with evidence.

ESC-005 Scope limits. The initial release MUST NOT claim universal compilation of arbitrary libraries, uniform accelerator access, coherent global memory or instantaneous state migration across vendors. Such capabilities require their own specification and validation.

The initial scope excludes an operating system, a replacement GPU driver stack, a universal numerical kernel language and a promise that any Python package runs on every terminal. Existing engines perform model execution through qualified adapters. General-purpose systems programming can be considered through later ADRs.

4 Logical and physical architecture

The trinitarian concept describes three logical roles. Their placement is configurable; a deployment may contain several nodes of each kind, and an authorised machine may host several roles.

Role	Responsibility
Interaction	Human interfaces, project inspection, submission and results
Coordination	Policy, planning, scheduling, delegation and durable task state
Execution	Authorised operations on local, cluster or remote resources

ARQ-001 Resource cardinality. Configuration MUST admit lists of nodes, execution groups, clusters and providers. It MUST NOT require either a Mac cluster or an NVIDIA cluster as a structural prerequisite. A deployment MAY contain zero instances of either family.

ARQ-002 Independent roles. A machine MAY exercise several authorised roles. Its brand or CPU architecture MUST NOT automatically determine its authority, trust or responsibility for a project.

ARQ-003 Technical capabilities. Resource selection MUST distinguish declared, verified and observed technical capabilities. A detected resource MUST NOT be represented as qualified for every model. Technical capability does not grant permission to use it.

ARQ-004 Control and data paths. Coordination MUST control authorisation and track operations without requiring all files, tensors or results to pass through the coordinator. Direct transfers MUST have their own authorisation and traceability.

ARQ-005 Resource managers. The system MUST support a standalone executor. It MAY integrate managers such as Kubernetes or Slurm through separate contracts. Authority over local reservations MUST be explicit to prevent incompatible allocation of the same resource. [F15] [F16]

ARQ-006 Execution groups. Resources used together for one inference or training operation MUST belong to a group validated for that execution mode. The availability of several destinations for independent tasks MUST NOT prove compatibility for joint computation.

Independent workflow stages can be placed on different vendors when their input and output contracts match. Splitting one model across those vendors requires a specifically qualified distributed engine. Control-plane portability and numerical-engine portability are separate claims.

5 Core language contracts

LNG-001 Specification. TriX MUST have a versioned specification for types, effects, errors, concurrency and distributed execution. The reference implementation MUST be testable against it.

LNG-002 Syntax. The approved grammar MUST define lexical rules, precedence, associativity and name resolution. Until approval, syntax examples MUST be labelled illustrative. A program MUST NOT change meaning merely because its hardware vendor or operating system changes.

LNG-003 Verification. The verified core MUST check types before execution. Local type inference MAY be used. Public interfaces MUST expose unambiguous types and contracts. Dynamic operations MUST cross an identifiable boundary.

LNG-004 External values. Values from models, files, external formats or extensions MUST be validated before use as typed values. Failed conversion MUST produce an identifiable error.

LNG-005 Absence and failure. The language MUST distinguish a present value, absence and failure. A remote failure MUST NOT implicitly become an empty successful result.

LNG-006 Mutability. Values SHOULD be immutable by default. Mutation MUST be explicit and follow documented ownership, isolation or synchronisation rules.

LNG-007 Resources. The language MUST define who controls resource lifetime, when resources may be copied, moved or shared, and how they are released. The final borrowing, reference-counting or other mechanism remains a bounded design decision.

LNG-008 Remote memory. A local address MUST NOT cross a process or network boundary as a still-valid reference. Transfers MUST use serialisable values or resource identifiers with verifiable validity and authorisation.

LNG-009 Effects. File, network, process, tool, model and remote-resource operations MUST declare relevant effects. The specification MUST identify static checks and checks performed at access time.

LNG-010 Authority capabilities. A declaration in a program or extension MUST NOT grant authority by itself. Effect execution MUST depend on capabilities granted and checked by the runtime's authorisation system.

LNG-011 Policy propagation. Data restrictions MUST survive functions, collections and task boundaries. Reducing restrictions MUST require an explicit, authorised and audited operation.

LNG-012 Resource declarations. The language MUST distinguish requirements, reservations, enforceable limits, objectives and preferences. Estimates MUST be labelled, and final selection MUST consider observed destination capacity.

6 Concurrency and interoperability

CON-001 Concurrency scopes. Concurrent constructs MUST define parent-child relationships, scope completion and propagation of failure and cancellation. Work surviving its original scope MUST require an explicit durable-execution operation.

CON-002 Shared state. The verified subset MUST specify safe sharing. Actors and messages MAY be used. Messages MUST obey types, authority and data-circulation rules.

CON-003 Cancellation. The language MUST distinguish cancellation requested, accepted and actually completed. An expired deadline MUST NOT prove that remote execution stopped.

CON-004 Streams. Result streams MUST define ordering, completion, failure and backpressure. Partial output MUST be distinguishable from accepted final output.

CON-005 Intermediate representation. Compilation or interpretation MUST produce a verifiable plan containing tasks, dependencies, types, effects and requirements. Its schema MUST be versioned independently of platform binaries.

CON-006 Unknown requirements. An executor unable to understand a mandatory requirement MUST reject the task before effects begin. An optional feature MAY be omitted only when the contract permits that alternative.

CON-007 Foreign languages. Rust, Python, Swift and other integrations MUST define representations, ownership, object lifetimes, errors, concurrency and cancellation. Process isolation SHOULD be used when extension failure could compromise the core.

CON-008 Guarantee boundary. Foreign code, native libraries and binary interfaces MUST NOT implicitly inherit verified-core guarantees. Direct interfaces MUST identify their ABI or protocol and compatible version combinations.

CON-009 Dependencies. Modules and extensions MUST have unambiguous identities, versions and dependencies. Installation MUST permit pinning the exact artifacts used. Importing a module MUST NOT increase authority.

CON-010 Reproducibility. Tasks MUST be able to declare exact identity, numerical tolerance or quality criteria as their reproduction objective. Relevant versions, configuration and seeds MUST be recorded. Those records alone MUST NOT be represented as proof of identical output.

7 Authority, security and audit

The reference governance profile preserves human authority and an independent audit line. Operational delegation runs from the human authority to the global Orchestrator, then to a project's SubOrchestrator and its agents. Project Auditors report to the Global Auditor, which reports to the human authority. This routing constrains autonomous agent decisions; authorised infrastructure transfers remain possible.

GOV-001 Human authority. TriX MUST recognise final human authority over policies, delegation and revocation. Work within current authorisation MUST be able to proceed autonomously without redundant confirmation requests.

GOV-002 Context and hierarchy. Each task MUST identify its project, issuing authority, scope and policy version. Delegation MUST remain within those bounds. In the reference profile, each project MUST have a SubOrchestrator reporting to the global Orchestrator and a Project Auditor reporting to the Global Auditor, which reports to the human administrator. Coordination MUST NOT subordinate the audit line.

GOV-003 Agent channels. In the reference governance profile, agents and subagents below the Orchestrator MUST NOT establish autonomous peer-to-peer communication. Delegations and results MUST follow authorised routes. Technical transfers between executors MAY occur without creating autonomous agent channels.

GOV-004 Independent audit. Auditor identities and permissions MUST be separate from supervised components. Operational credentials MUST NOT authorise modification or deletion of audit evidence or auditor permissions. Authorised administrative changes MUST follow a separate procedure and create verifiable evidence under the published trust model.

GOV-005 Enforced policy. Authority MUST be checked at relevant access boundaries. Each profile MUST define revocation propagation, cached-authorisation validity and offline behaviour. A received, validated revocation MUST block new affected effects and trigger handling of ongoing work. Offline execution MUST NOT extend previously granted scope or validity.

GOV-006 Replicated coordination. Replication MUST prevent competing coordinators from causing incompatible accepted allocations or effects over the same resource. Commands carrying stale authority MUST be rejected at the identified enforcement boundary. Recovery MUST preserve work identity and authority.

GOV-007 Untrusted content. Instructions in documents, model responses and tool outputs MUST NOT become executable policy by themselves. Received content and granted authority MUST remain distinct.

GOV-008 Component identity. Nodes, coordinators, clients and providers MUST be authenticated on relevant connections. Transport MUST protect integrity and confidentiality under the stated trust model. A node name or task identifier MUST NOT act as an access credential.

GOV-009 Secrets and trust. Credentials MUST remain outside programs, plans and records not authorised to contain them. The specification MUST publish trust assumptions, including administrator power and isolation limitations.

GOV-010 Assurance boundaries. The trusted computing base MUST be identified for each assurance claim, including the verifier, enforcement and state components relied upon and relevant host, transport and compilation dependencies. Structural plan verification MUST NOT be treated as proof of source-to-plan semantic preservation. A compromised privileged host may defeat local isolation; the trust model MUST identify which independently controlled audit systems remain trustworthy in that case.

8 Task contracts and failure handling

TAR-001 Contract. Each task MUST declare inputs, outputs, permitted effects, requirements, completion criteria and unavailability behaviour. Models, tools and policies MUST have a version or an explicit resolution rule.

TAR-002 Identity. Workflows, tasks, attempts and external effects MUST have distinguishable identities. A new attempt MUST NOT erase history or create implicit new authority.

TAR-003 States. The system MUST distinguish at least created, validated, queued, running, cancelling, succeeded, failed, cancelled and awaiting reconciliation. Reserved, waiting or suspended states MAY be added with defined semantics. Transitions MUST be validated and recorded.

TAR-004 Completion. Success MUST require verification of the output contract and known effects. A missing response with an uncertain outcome MUST trigger reconciliation when the result cannot otherwise be determined.

TAR-005 Retries. Retry policy MUST distinguish repeatable computation, idempotent effects and effects requiring reconciliation. TriX MUST NOT promise exactly-once execution for arbitrary external effects.

TAR-006 Deduplication. Repeatable effects MUST use identification, deduplication or equivalent destination-appropriate mechanisms. Where these do not exist, the task MUST follow an explicit verification or intervention policy.

TAR-007 Actual cancellation. The executor MUST report whether cancellation is cooperative, stage-dependent or achieved by process interruption. Completed effects MUST remain recorded and MUST NOT be considered undone without confirmation.

TAR-008 Checkpoints. A checkpoint MUST identify content, versions, resumption unit and compatibility conditions. Logical session state MUST NOT be confused with transferable GPU, process-memory or KV-cache state.

TAR-009 Offline continuity. Each workflow MUST define loss-of-connectivity behaviour. Offline operation MUST NOT increase authority, export data through an alternative channel or create competing authority over shared resources.

TAR-010 Provenance. Execution MUST preserve enough evidence to identify code, policies, models, inputs, attempts and results. Recorded content MUST obey access and retention policies.

TAR-011 Recovery. Each operational profile MUST define recovery objectives, possible progress loss and reconciliation mechanisms. Unmeasured objectives MUST be labelled targets, not operational guarantees.

9 Data, models and memory

DAD-001 Classification. Data and artifacts MUST identify their project, permitted access, authorised destinations and retention policy. These properties MUST be checked during planning and actual access.

DAD-002 Derived data. Excerpts, summaries, embeddings, transcripts, extracted images and intermediate results MUST retain applicable source restrictions. Reclassification or export MUST be explicit and authorised. Transformation alone MUST NOT be described as automatic anonymisation.

DAD-003 Model identity. A model MUST be identified to the precision required by the task, including weight revision, tokenizer, format, quantisation, configuration, context and capabilities where relevant. A commercial name alone MUST NOT replace a more precise identity required for reproduction.

DAD-004 Substitution. Changes to model, quantisation, precision or implementation MUST respect authorised alternatives and quality criteria. Spare capacity or lower cost MUST NOT justify silent substitution.

DAD-005 Multimodal data. Multimodal inputs and outputs MUST declare relevant formats and limits, including dimensions, duration or volume where applicable. Conversions MUST retain provenance and disclose task-relevant losses.

DAD-006 Memory accounting. Admission MUST consider relevant weights, caches, buffers, temporaries and operational margin. Estimates MUST distinguish CPU memory, accelerator memory and memory actually shared through a unified architecture.

DAD-007 Caches. Cache reuse MUST respect model, configuration and data authorisation. Caches MUST NOT provide unauthorised access between projects.

DAD-008 Distribution. Artifact location, transfers and replicas MUST be explicit. Aggregated memory across nodes MUST NOT be presented as coherent memory available to any process.

DAD-009 Consistency. The specification MUST define consistency for shared project, artifact and metadata changes. Results under different revisions MUST remain distinguishable. Write conflicts MUST have a documented resolution rule.

DAD-010 Retention and deletion. The product MUST support retention and deletion policies. It MUST distinguish logical removal, access revocation and physical-erasure guarantees actually supported by the environment.

Apple unified-memory execution, Core ML device selection and distributed GPU communication have different boundaries. A plan must qualify the engine and execution group actually used. No reference listed here makes one host's memory an automatically coherent extension of another host. [F09] [F10] [F20] [F21]

10 Resources, priorities and placement

REC-001 Decision order. Prohibited, incompatible or resource-ineligible destinations MUST be excluded before optimisation. Priority, price or performance MUST NOT compensate for a violated mandatory requirement.

REC-002 Limit enforcement. A limit MUST be described as enforced only when an appropriate mechanism has been verified. Accounting reservations or usage observations MUST NOT be described as physical isolation.

REC-003 Admission. Admission MUST account for quotas, ongoing work and usable capacity. Policies SHOULD protect interactive capacity and prevent indefinite starvation of eligible work.

REC-004 Network and locality. Planning MUST consider transfers, latency, bandwidth, data availability and setup costs relevant to the whole workflow.

REC-005 Shared capacity. Containers, virtual machines and subsystems exposing the same equipment MUST share a physical-resource identity or equivalent coordination to prevent double counting and over-reservation.

REC-006 Mobility and power. Terminals MUST advertise availability, power state and relevant system restrictions. Participation MUST respect user policy and actual ability to continue in the background.

REC-007 External costs. Remote operations MUST have a budget and admission policy. Estimates, confirmed charges and uncertain liabilities MUST be distinguishable. A provider's inability to impose a strict cap on an in-flight request MUST be disclosed.

REC-008 Placement explanations. The system MUST explain why it selected or excluded a destination without exposing secrets or other projects' data. No eligible destination MUST result in waiting, an error or a previously authorised alternative.

Decision layer	Example checks
Authority and data	Project scope, destination permission, credential validity
Compatibility	Model operation, engine, precision, ABI and mandatory features
Admission	Memory, quotas, reservations, concurrency and available budget
Optimisation	End-to-end latency, cost, energy and local user preferences

11 Platform identity and compatibility

CPU instruction set, vendor, operating system, ABI, accelerator and execution environment are separate dimensions. In this document, x86-64, x64 and AMD64 name the same 64-bit instruction-set family; x86-32 identifies its 32-bit predecessor. ARM64/AArch64 is a different family. Intel Arc is an accelerator product family, not a CPU instruction set. A vendor name alone is never a complete target identifier. [F04] [F05] [F06]

PLT-001 Target identity. Each package and executor MUST declare the relevant platform combination. Generic labels such as Linux, Windows, ARM or GPU MUST NOT constitute the final compatibility declaration.

PLT-002 Observed architecture. Inventory MUST distinguish physical architecture, process architecture and native, emulated or virtualised execution. Optional CPU extensions MUST be checked before use.

PLT-003 Minimum versions. Each release MUST distinguish client, local-execution and feature-specific minima. A compiler's minimum target MUST NOT automatically become the complete product's minimum.

PLT-004 Dependencies. Python environments and native libraries MUST have pinnable dependencies and compatibility declared by OS, architecture and ABI. Reproducible adapter-specific isolation is RECOMMENDED. [F23]

PLT-005 Linux and subsystems. Distributions, system libraries and environments such as WSL MUST have separate profiles where they affect execution or acceleration. A container image MUST NOT replace host-driver and resource verification. [F24]

PLT-006 Client versus executor. Interface support, remote control, local CPU execution, acceleration and distributed computation MUST be classified separately. A fully supported control client MAY lack general local execution.

PLT-007 Mobile systems. iOS, iPadOS and Android MUST have explicit lifecycle and availability profiles. An installed client MUST NOT be advertised as a permanent executor without supporting mechanisms and conditions. [F07] [F08]

PLT-008 Physical validation. Cross-compilation and emulation MAY validate parts of the core. Acceleration, energy and performance claims MUST be validated on representative physical hardware.

PLT-009 New platforms. New targets MUST be addable through platform interfaces and applicable adapters. The central grammar MUST NOT create incompatible vendor-specific dialects.

Target family	Initial qualification intention
macOS on Apple silicon	Native client and local executor; qualify each engine and machine
Linux x86-64 and ARM64	Server and terminal executors; qualify distribution, ABI and drivers
Windows x86-64 and ARM64	Client and executor profiles separately; distinguish emulation and WSL
iOS, iPadOS and Android	Control clients first; local execution subject to lifecycle and device policy
Browser	Web control client; local computation only through a separately qualified browser profile
x86-32, RISC-V, BSD and other targets	Extension candidates; no qualification claim in this edition

These are planning categories. No row certifies a platform or sets a minimum operating-system version. Actual qualification records must pin a complete tested combination.

12 Accelerators, managers and remote providers

INT-001 Separate interfaces. The design MUST separate platform, accelerator discovery, execution engine, cluster manager and remote-provider interfaces. One extension MAY implement several interfaces while preserving their responsibilities.

INT-002 Minimum execution contract. Task-execution adapters MUST expose identity, capabilities, request validation, submission, observation, results and cancellation requests. Other interfaces MUST publish contracts appropriate to their responsibility. Semantics and limitations MUST be explicit; unavailable operations MUST NOT be advertised as supported.

INT-003 Optional operations. Reservation, suspension, checkpointing, resumption, state export, priorities and process groups MUST be negotiated. An absent capability MUST NOT be simulated by claiming a stronger guarantee than the implementation provides.

INT-004 Third-party extensions. Contract specifications, an SDK, examples and conformance tests MUST be available. A third party MUST be able to add an integration without changing central language semantics.

INT-005 Origin and trust. Extensions MUST identify maintainer, version, dependencies and required permissions. Installation and update MUST verify integrity and the defined trust policy for that origin.

INT-006 Model validation. Recognising a device or loading an engine MUST NOT qualify every model. Each advertised capability MUST identify its tested scope.

INT-007 Remote models. Model APIs MUST obey the same task, data-location, credential, cost and audit contracts. A model-proposed tool call MUST return through local authorisation before execution.

INT-008 Isolation. Adapters MUST operate within the authority of assigned work. Failure or restart MUST produce reconcilable errors and states. Separate processes and dependency environments SHOULD reduce interference between engines.

INT-009 Joint computation. Splitting one inference or training operation across vendors MUST have a specific backend and tests for operations, communication, precision and recovery. Independent task support MUST NOT be advertised as arbitrary-device tensor parallelism.

Integration family	Candidate path and qualification boundary
Apple	MLX, Metal-facing engines or Core ML; qualify API, hardware and model [F09] [F10]
NVIDIA	CUDA-based engines; qualify host architecture, driver, engine and model [F11]
AMD	ROCm or other maintained engines; qualify the exact device and software matrix [F12]
Intel	OpenVINO and device-specific engines; Gaudi uses its own software stack [F13] [F14]
Qualcomm	QNN-compatible integrations; qualify OS, device, model and operator coverage [F17]
Other accelerators	TPU/JAX, AWS Neuron or future adapters under their own contracts [F18] [F19]
CPU	Native engine appropriate to the task; verify architecture and optional instructions
Remote providers	Versioned API adapter with declared residency, rate limits, cost and model identity

Kubernetes and Slurm are resource managers rather than model engines. PJRT is an example of an extensible device interface, not a universal compatibility guarantee. TriX adapters must keep these layers distinct. [F15] [F16] [F22]

13 Versions, distribution and updates

VER-001 Version axes. Language, plan, protocol, extension API and persistent-state versions MUST be identifiable separately from the product release.

VER-002 Negotiation. Components MUST check applicable versions and capabilities before relevant execution. Unknown mandatory features MUST cause explicit refusal. Partial operation MAY occur only where its contract defines it.

VER-003 Releases. Each release MUST include target, dependency, integrity, origin and support manifests. Developer tools, clients, executors, adapters and models MUST be independently distributable where their contracts permit it.

VER-004 Updates. Updates MUST check compatibility, handle in-flight work and confirm operation before final activation. Gradual activation is RECOMMENDED for execution-affecting changes.

VER-005 Rollback. Rollback MUST account for persistent state and checkpoints. Reinstalling an old binary MUST NOT be presented as valid recovery if it cannot interpret the recovered state.

VER-006 Support window. Concurrent-version support MUST be explicit and tested. A rule such as current and previous version MAY be claimed only after specifying which components, versions and features it covers.

VER-007 Stability. Breaking changes MUST identify impact, migration and treatment of existing programs. Target deprecation MUST state consequences and available alternatives without silently changing task requirements.

VER-008 Support labels. The product MUST distinguish project-qualified, maintainer-qualified, qualified with limitations, experimental, remote-client-only and unsupported combinations. Labels MUST apply to specific features and version combinations.

VER-009 Update authenticity. Published packages MUST use signatures or equivalent channel-appropriate authenticity. Verification MUST respect authorised keys, revocation and version policy. A valid signature on an old package MUST NOT alone authorise downgrade.

BLD-001 Build process. Each supported target MUST have a documented, automatable build process identifying tools, SDKs and prerequisites. Restrictions preventing independent reconstruction MUST be documented.

BLD-002 Component inventory. Distributions MUST preserve an inventory of components, versions, hashes and licences. OS, architecture or ABI variants MUST NOT implicitly share the same binary-dependency set.

BLD-003 Artifact reproduction. The project MUST check reconstruction in a clean environment. Where byte identity cannot be guaranteed, it MUST declare the verified equivalence level and retain artifact provenance.

Build provenance and artifact verification should be designed together. The SLSA specification provides a reference vocabulary; citing it does not establish that TriX achieves a SLSA level. [F29]

14 Developer tools and user experience

DEV-001 Essential tools. The first usable toolset MUST validate programs or plans, inspect capabilities, run work, observe progress, request cancellation and explain errors. Consistent formatting and editor integration are RECOMMENDED.

DEV-002 Diagnostics. Errors MUST identify the operation, source location when available, violated requirement and known cause. They MUST distinguish program errors, policy refusal, incompatibility, unavailability and execution failure.

DEV-003 Preview. The platform MUST inspect a plan without performing its external effects. Preview MUST expose relevant requirements and uncertainties and MUST NOT describe unallocated resources as reserved.

DEV-004 Documentation. Every stable feature MUST have semantic documentation, executable examples, relevant errors and support conditions. User-facing executable examples MUST be checked against their stated versions.

DEV-005 Observability. The product MUST expose states, attempts, timings and resource usage. Telemetry and logs MUST minimise private data and respect project permissions.

DEV-006 Shared interface contract. Clients on different platforms MUST share task, action and state contracts. Visual implementations MAY be platform-specific. Keyboard access, readable messages and status indications independent of colour are RECOMMENDED.

DEV-007 Inspectable dependencies. Tools MUST expose effective execution versions and permissions. Installation diagnostics SHOULD distinguish installed package, detected device, usable engine and qualified model.

DEV-008 Complete examples. Documentation MUST include local, cluster and authorised remote-provider workflows and a policy-refusal case. Examples MUST show unavailability and failure behaviour.

UXD-001 Web and native access. The product design MUST provide a web control application and permit native clients using the same versioned service contracts. A browser client MUST NOT silently obtain native filesystem, process or accelerator authority; any companion bridge MUST be explicitly enrolled and restricted.

UXD-002 Accessibility and localisation. User interfaces MUST publish an accessibility target and test keyboard operation, focus, screen-reader names and errors. The web reference profile SHOULD target WCAG 2.2 AA. Interface translations MUST preserve operation meanings, units and authority prompts; diagnostic codes MUST remain stable. [F30]

UXD-003 Explainable autonomy. A client MUST show active delegation scope, relevant limits and task state. Previously granted authority MUST remain usable without repeated prompts. A change requiring more authority MUST identify the concrete new action and reason before requesting it.

UXD-004 Decision traces. Placement and policy traces MUST record considered constraints and a redacted reason for each decision. Telemetry exports MUST use a declared schema mapping and minimise sensitive payloads. OpenTelemetry conventions MAY be used with pinned versions or commits; changing upstream conventions MUST NOT silently alter TriX evidence fields. [F28]

15 Language and compiler specification

Baseline choices for adoption

Adopt the following as the version 0.2 planning baseline, subject to the project's normal adoption record.

1. **Initial language scope.** The first executable subset describes typed task graphs, pure data transformations, dependencies, conditions and execution contracts. Existing adapters perform model inference, tools and substantial numerical computation.

2. **Two distinct milestones.** The initial SDK and serialized plan establish the execution contract. The later source frontend establishes the programmer-facing grammar. Delivering the SDK alone does not constitute delivery of the complete source language.

3. **Small value model.** The first evaluator uses immutable values, explicit optional and result values, checked integers, exact finite decimal values and typed quantities. Tensor arithmetic and binary floating-point computation remain engine operations in this baseline.

4. **Opaque resources.** External artifacts are referenced through typed identifiers. Live allocations and access authority use separate scoped handles. The baseline exposes no raw pointers or arbitrary shared mutable memory.

5. **Verified admission.** Every submitted plan crosses the same plan-verification boundary, regardless of its frontend or signature. Target binding and authorization are checked again by the execution system where required.

6. **Explicit build inputs.** Locked dependency resolution is the release default. Planning receives explicit inputs and does not implicitly read clocks, environment variables, credentials or the network.

These decisions specify observable behavior. They do not select final keywords, file extensions, a public package registry, the complete logical-plan encoding or the implementation language of every component.

Requirements for the first executable subset

LSP-001 Closed baseline profile. The specification MUST publish an exhaustive list of supported value types, expressions, graph constructs, built-in operations and effects for the first subset. Unlisted constructs MUST be rejected or identified as extensions requiring a different declared profile. An extension MUST NOT silently alter baseline semantics.

LSP-002 Bounded composition. The baseline MUST support acyclic task dependencies, explicit conditional selection and finite collection processing with declared expansion limits. It MUST reject dependency cycles and unbounded plan expansion. General recursion, unrestricted metaprogramming, user-defined kernels and arbitrary native calls are outside this baseline; later support requires an additional specification.

LSP-003 Frontend boundary. An SDK builder, declarative input and future TriX source frontend MUST produce plans governed by the same semantics. Host-language builder code MUST NOT be treated as verified TriX code. Executing a Python or other builder requires a separately declared build environment and permissions before the resulting plan can be verified.

LSP-004 Source specification gate. Before a source frontend is stable, its lexical rules, grammar, scope, name resolution, evaluation rules and source-to-plan mapping MUST be published together. The final grammar MAY be selected later. Syntax demonstrations MUST remain labelled illustrative until accepted into that specification and its conformance corpus.

LSP-005 Baseline data types. The baseline MUST define Boolean, UTF-8 string, byte sequence, signed 64-bit integer, exact finite decimal, typed quantity, immutable record, ordered list, string-keyed map, tagged union, optional value and result value. Each type MUST define construction, equality and serialization. Model tensors MUST declare an external representation and element type rather than inheriting scalar evaluator rules implicitly.

LSP-006 Explicit conversions. The evaluator MUST NOT apply implicit string-to-number conversions, truthiness conversions or lossy numeric narrowing. Invalid input, absent values and failed operations MUST remain distinguishable. Conversions that can fail MUST return a typed failure. Accepted runtime values MUST conform to the same value rules as source literals.

LSP-007 Numeric behavior. Integer overflow, division by zero and invalid numeric conversion MUST produce specified errors. Decimal addition and multiplication MUST be exact within declared representation limits; exceeding those limits MUST fail explicitly. Decimal division requiring rounding MUST specify scale and rounding mode. Binary floating-point and tensor operations MUST execute under an explicit engine contract outside the baseline scalar evaluator.

LSP-008 Units and budgets. Durations, byte quantities, counts and monetary amounts MUST be distinguishable types. Arithmetic MUST reject incompatible units. A monetary amount MUST retain its currency identifier; conversion requires an explicit conversion input. Encodings MUST define base units and scaling, and reject negative or out-of-range values where the field's contract excludes them.

LSP-009 Text and ordering. Baseline strings MUST be compared by their exact Unicode scalar sequences, without implicit normalization or locale-sensitive comparison. Duplicate map keys MUST be rejected. Map iteration MUST use a documented stable ordering; insertion order MUST NOT become an undeclared dependency. Lists preserve order. Identity and ordering rules MUST remain independent of the host filesystem and locale.

LSP-010 Evaluation order. The specification MUST define evaluation order and conditional short-circuiting for pure expressions. Task ordering MUST follow explicit dependencies rather than source layout alone. Parallel completion order MUST NOT alter ordered results unless the operation explicitly declares arrival order. Optimizations MUST preserve these observable rules and all required effects.

Requirements for resources and effects

LSP-011 Resource reference model. Immutable artifact references MUST identify an object and its representation without themselves granting access. Live allocation handles MUST have an owner, scope, release operation and expiry or invalidation behavior. Copying a resource identifier MUST NOT duplicate an allocation or authorization. The baseline MUST prohibit raw addresses in portable values.

LSP-012 Resource consumption. Operations that transfer or release exclusive handles MUST consume the corresponding authority exactly as defined by the handle contract. Reuse after confirmed release MUST fail. Failure paths MUST trigger documented cleanup or reconciliation. This is a resource-lifecycle rule; it MUST NOT be presented as a guarantee of immediate cleanup after a disconnected worker fails.

LSP-013 Effect composition. Each built-in operation and callable module interface MUST expose a machine-readable effect summary. A composed operation MUST conservatively include the effects of its callees and possible branches. A wrapper MUST NOT conceal network access, process creation, model invocation, time access or randomness. Effect declarations describe possible actions and do not grant permission to perform them.

LSP-014 Pure planning boundary. Parsing, type checking and evaluation of baseline pure expressions MUST NOT perform task effects. Clock, randomness, environment lookup, filesystem discovery and remote inspection MUST be explicit inputs or separately authorized operations. Package acquisition MUST be a visible preparation step. Plan generation MUST NOT run dependency initialization code merely because a declaration is imported.

LSP-015 Conservative policy propagation. Within baseline verified transformations, an output MUST inherit applicable restrictions from all supplied inputs, including predicates controlling that output. A narrower classification requires the existing explicit reclassification contract. Foreign code MUST use a declared information-flow boundary; the system MUST NOT infer unrestricted output from missing provenance. This rule does not claim elimination of all covert channels.

LSP-016 FFI admission and containment. Baseline foreign integrations MUST use versioned messages or a documented restricted calling boundary, with validated lengths, representations, ownership and errors. An in-process native extension requires a separately declared trusted profile. Foreign exceptions, panics or equivalent nonlocal exits MUST NOT cross a boundary that does not define them. Native-code access remains subject to the stated trust assumptions.

Requirements for plans and compiler trust

LSP-017 Logical and bound plans. The system MUST distinguish a portable logical plan from its execution binding. The logical plan records meaning, requirements and authorized alternatives. Binding records chosen artifacts, model variants, adapters, targets and relevant policy decisions. Late-bound choices MUST produce a resolution record before their effects; they MUST NOT silently mutate the original plan's meaning.

LSP-018 Plan invariants. Before admission, a separately callable verifier MUST check schema versions, unique node identifiers, reference resolution, graph validity, type compatibility, mandatory fields, declared effects and structural resource bounds. It MUST reject ambiguous encodings, duplicate fields and unsupported mandatory constructs. Validation MUST finish before loading task plugins or starting task effects.

LSP-019 Canonical identity. The plan specification MUST define canonical semantic encoding and a content digest. Identical source, declared inputs, resolved language dependencies, compiler version and compilation options MUST produce the same logical plan identity. Comments, timestamps, machine paths and display metadata MUST NOT change that identity unless explicitly semantic. Execution attempts and authorizations MUST retain separate identities; a content digest is not an access credential.

LSP-020 Verification limits. Plan verification MUST state what it proves and what remains for runtime enforcement. It cannot certify arbitrary adapter behavior, available memory, current permission or future numerical quality merely from a manifest. A compiler signature or trusted origin MUST NOT bypass verification. Source-to-plan semantic preservation MUST be validated independently of structural plan validity.

LSP-021 Compiler transformations. Each optimization MUST preserve data dependencies, errors, effect ordering, policy restrictions and observable results under the declared semantic profile. Transformations that change model, precision, provider or permitted outcomes MUST be explicit planning decisions governed by the task contract, rather than invisible compiler optimizations. An unoptimized reference path MUST remain available for conformance comparisons.

Requirements for packages and builds

LSP-022 Module resolution. Imports MUST resolve through declared namespaces and package roots. Ambiguous names, undeclared dependencies and traversal outside approved roots MUST be rejected. Platform-specific implementations MAY share a module contract but MUST identify their target artifacts. Loading an interface declaration MUST NOT execute module initialization or native code.

LSP-023 Lockfile contract. A release lockfile MUST record the resolved transitive dependency graph, immutable content identities, artifact origins, selected target variants and relevant toolchain identities. Locked builds MUST fail on missing or mismatched artifacts rather than silently selecting alternatives. Permission requirements MAY be recorded for inspection; recording them MUST NOT grant them.

LSP-024 Controlled build inputs. Release builds MUST declare allowed environment inputs, tool invocations and generated sources. Once declared artifacts are available, verification and construction SHOULD operate without undeclared network access. Required exceptions MUST be recorded in provenance. Native build scripts and SDK builders MUST run with documented authority and MUST NOT be assumed harmless because they run before deployment.

LSP-025 Separation of caches and trust. Build and compilation cache keys MUST include all inputs that affect generated semantics or target compatibility. Cache reuse MUST verify artifact integrity and the receiving project's access rights. A hit MUST NOT skip required plan verification or current authorization checks. Secret values MUST NOT become public cache identifiers or diagnostics.

Requirements for developer tools and acceptance

LSP-026 Structured diagnostics. The compiler and verifier MUST emit stable machine-readable error identifiers, severity, phase, relevant plan node and available source locations. Generated code MUST retain a source map to the originating declaration where possible. Human-readable documentation and canonical messages MUST be English; localized presentation MAY coexist without changing error identity or meaning.

LSP-027 Inspectable compilation. Developer tools MUST expose the normalized logical plan, dependency resolution, effect summary and diagnostics without submitting work. Formatting or source migration tools MUST preserve meaning, or display an explicit semantic change requiring review. Inspection MUST distinguish unresolved choices from fixed bindings and unmeasured estimates from enforced limits.

LSP-028 Conformance corpus. The language release MUST include positive and negative examples covering each baseline construct and normative edge case. The corpus MUST define accepted values, rejected programs, expected diagnostics and permitted observable traces. At least one reference evaluator or unoptimized execution path MUST support comparison with the production implementation for the verified subset.

LSP-029 Stability gate. A source-language profile MUST NOT become stable while its grammar, value semantics, source-to-plan mapping or required diagnostics remain unspecified. A plan-only profile MUST define its complete plan encoding, value semantics, semantic invariants and diagnostics before becoming stable; it MUST NOT claim source-language conformance. A release MUST pass its conformance corpus on each claimed implementation target and test malformed or oversized inputs without crashes or unbounded verifier work. Changes to accepted meanings require an identified compatibility decision and updated corpus.

16 Runtime and operational specification

Runtime records and state transitions

RUN-001 Authoritative record. Each task MUST have a durable record containing tenant, project, principal, workflow, task and attempt identities; plan and input revisions; authority and policy references; controller epoch; lifecycle state; cancellation intent; output references; and effect records. Updates MUST use an atomic version check or equivalent serialization. Duplicate and reordered messages MUST NOT overwrite newer accepted state.

RUN-002 Separate status dimensions. The runtime MUST expose the following dimensions independently. A connection failure MUST NOT become evidence that execution stopped or an effect did not occur.

Dimension	Proposed baseline values	Meaning
Task lifecycle	CREATED, VALIDATED, QUEUED, RUNNING, CANCELLING, RECONCILING, SUCCEEDED, FAILED, CANCELLED	The coordinator's durable workflow decision
Attempt execution	NOT_STARTED, STARTING, ACTIVE, STOP_REQUESTED, STOPPED	Last evidenced execution condition, with observation time and source
Observation knowledge	CONFIRMED, STALE, UNKNOWN, CONFLICTING	Confidence and freshness of the particular observation
Individual effect outcome	NOT_DISPATCHED, UNKNOWN, APPLIED, NOT_APPLIED	Evidence about an external action; compensation is a separate action

CONFIRMED means supported by identified evidence within the relevant validity conditions. It does not imply continuous connectivity. A task can be RECONCILING while its last execution observation is ACTIVE and STALE. Effects already known to be APPLIED remain recorded after cancellation.

RUN-003 Guarded transitions. The baseline task lifecycle MUST follow the following transition contract. RUNNING includes an attempt durably authorized for dispatch; actual computation is represented separately. Reservations and suspension MAY extend the model only with documented transition guards and recovery behavior.

From	To	Required guard
CREATED	VALIDATED	Plan, types, identities and requested authority pass validation

From	To	Required guard
VALIDATED	QUEUED	Queue admission accepted; this alone does not reserve hardware
CREATED, VALIDATED, QUEUED	FAILED	Validation, admission or deadline failure; no unresolved dispatch
CREATED, VALIDATED, QUEUED	CANCELLED	Cancellation accepted; no current or unresolved dispatch; prior effects accounted for
QUEUED	RUNNING	Current admission and authority checked; attempt, allocations and dispatch intent durably recorded
RUNNING	QUEUED	Previous attempt settled or adequately fenced; retry remains safe, authorized and within budget
RUNNING	CANCELLING	Durable cancellation intent recorded before requesting executor cancellation
RUNNING, CANCELLING	RECONCILING	Execution or effect outcome cannot be established sufficiently
RECONCILING	RUNNING, CANCELLING	Evidence identifies the same active attempt; durable cancellation intent determines the destination
RECONCILING	QUEUED	Safe retry established; no cancellation intent; authority and admission revalidated
RUNNING, RECONCILING	SUCCEEDED	Output and effect obligations verified; unresolved conflicting attempts excluded
RUNNING, CANCELLING, RECONCILING	FAILED	Failure established, retry policy exhausted, relevant effects reconciled and any terminal-decision race resolved
CANCELLING, RECONCILING	CANCELLED	Cancellation disposition established; outstanding effects resolved and retained in the record
CANCELLING	SUCCEEDED	Verified completion won the explicitly serialized completion/cancellation race

RUN-004 Terminal decisions. Completion and cancellation MUST have a documented serialization point. An acknowledgement that cancellation was requested MUST NOT promise that cancellation won. A deadline expiry MUST NOT imply remote termination. CANCELLED MUST identify the confirmed disposition: execution stopped, or an explicitly permitted alternative that blocks further managed effects while separately tracking ongoing execution and cost. It MUST NOT hide unresolved effects or imply reversal. Terminal decisions MUST preserve evidence and history; contradictory late evidence MUST create a visible correction and incident, prevent unsupported reliance on affected results, and never silently rewrite the past. Explicit follow-up work receives a linked task identity.

RUN-005 Workflow boundaries. Plans MUST define dependency success conditions, failure propagation, child-task ownership and permitted continuation after parent cancellation. Downstream tasks MUST NOT consume uncommitted results as final outputs. Progressive processing MUST declare its partial-result semantics. Fan-out, recursion, model/tool loops and spawned work MUST remain within explicit policy bounds. Agent delegation continues through the authorized hierarchy.

RUN-006 Event evidence. Transitions MUST retain event identity, task version, actor, authority epoch, causal references and evidence location. Wall-clock timestamps alone MUST NOT determine distributed event order. Observation freshness, event retention and replay behavior MUST be documented. Replayed notifications MUST NOT repeat task effects.

Leases, fencing and recovery

RUN-007 Allocation authority. Every exclusive allocation MUST identify its resource scope, owner, validity and monotonically ordered authority epoch. Replicated controllers MUST obtain conflicting ownership decisions through one serialization authority. A single-controller deployment MUST preserve equivalent exclusivity across restart. Controller election alone MUST NOT be treated as proof that previous work can no longer affect resources.

RUN-008 Enforcement of stale authority. Effect gateways, writable resource boundaries and artifact publication MUST reject stale fencing epochs where fencing is claimed. Checking only the controller's view is insufficient. If a backend cannot fence an accepted operation, the runtime MUST retain that operation as potentially active until reconciled; it MUST NOT claim that a new controller or expired lease revoked the already accepted effect.

RUN-009 Lease validity. Profiles MUST specify renewal, expiry, allowed clock uncertainty and the behavior of disconnected executors. Uncertain or expired authority MUST block new effects requiring that authority. Local monotonic timers MAY support lease enforcement under declared assumptions. Loss of connectivity MUST NOT extend a lease. Independently authorized offline work MUST use its own bounded allocation and delegation.

RUN-010 Controller recovery. Recovery MUST reconstruct tasks, attempts, allocations, budget reservations and effects from durable evidence before issuing replacement work. Heartbeat expiry MAY make a resource unavailable for new assignments; it MUST NOT prove that its process stopped. During a partition, exclusivity requirements take precedence over making conflicting assignments available. Unresolved operations remain visible.

Effects, retries and transaction boundaries

RUN-011 Effect ledger. Before an external request can escape, the runtime MUST persist an effect identity, intent, destination, payload identity and authorization, and mark its outcome potentially UNKNOWN. Repeated submissions of the same logical effect MUST preserve its deduplication identity across attempts. Reuse of that identity with a different payload MUST be rejected. The destination's deduplication scope, retention window and result-query behavior MUST be recorded.

RUN-012 Retry admissibility. Retry policies MUST bound attempts, elapsed time and resource or financial exposure. Each retry MUST revalidate authority and compatibility. Automatic retry of an uncertain effect is permitted only when the destination's semantics or reconciliation evidence make repetition safe. Switching providers MUST NOT silently convert an uncertain old effect into a fresh independent effect. Exhaustion of a deduplication window requires renewed reconciliation.

RUN-013 Qualified guarantees. Any exactly-once claim MUST name the transaction domain, failure model and enforcement mechanism. A durable local transaction or outbox alone MUST NOT be presented as exactly-once execution at an arbitrary external destination. Compensation MUST be a separately authorized, identifiable action with its own possible failure and uncertain outcome; compensation does not erase the original effect or establish complete rollback.

Data and artifact lifecycle

RUN-014 Publication contract. Artifacts MUST distinguish STAGED, VERIFIED, COMMITTED, TOMBSTONED, PURGE_PENDING and PURGED. Normal publication follows STAGED → VERIFIED → COMMITTED. COMMITTED requires validated content identity and atomic visibility of the output reference. Readers MUST reject incomplete or unauthorized artifacts. Abandoned staging data MUST have a cleanup policy. Content integrity alone MUST NOT confer access authority.

RUN-015 Retention inventory. Each artifact MUST carry ownership, classification, provenance, storage locations, authorized readers, retention policy and applicable holds. The inventory MUST cover replicas, checkpoints, caches, indexes, embeddings, transcripts, thumbnails and other derivatives. Shared storage MUST distinguish a project's reference from shared bytes still legitimately retained for another authorized owner. In-flight readers and references MUST be considered before reclamation.

RUN-016 Deletion semantics. Deletion MUST distinguish denial of future access, tombstoning, physical purge requests and verified completion within each storage domain. The runtime MUST report inaccessible or disconnected copies and external-provider limitations. It MUST NOT claim recall of bytes already disclosed to an authorized recipient. Retention expiry or deletion MUST prevent prohibited new reuse, including regeneration from retained derivatives, according to the effective policy and revocation window.

RUN-017 Backups and restore. Backups MUST declare coverage, encryption/key authority, retention, recovery compatibility and possible data loss. Restore MUST reconcile deletion, revocation and policy records before restored data becomes accessible. A recovery environment unable to establish current policy MUST remain restricted. Backup expiry, cryptographic erasure and physical deletion MUST be described separately, with their actual scope and evidence. Restore drills MUST include artifact content and metadata consistency.

Identity, tenancy, admission and budgets

RUN-018 Identity boundaries. Tenant, project, human, service, agent, device and workload identities MUST remain distinguishable. A single-user installation MAY have one tenant but MUST still define its isolation boundary. Task identifiers, artifact digests and node names MUST NOT function as credentials. Authorization MUST bind subject, action, resource, project, delegation scope and validity at each relevant access boundary.

RUN-019 Policy context. An admitted task MUST retain the policy version used for its decision and obey the documented effect of later revocation or policy changes. Delegated credentials MUST grant only the applicable workload scope; model outputs and external services MUST NOT enlarge it. Audit identities and permissions MUST remain separate from supervised operational identities. Cross-project sharing requires an explicit policy decision.

RUN-020 Scheduling contract. Admission MUST account for current allocations, queue bounds and relevant CPU, accelerator, memory, storage, transfer and concurrency demands. Policies MUST distinguish eligibility, priority, fairness, preemption and deadlines. Preemption MUST declare whether it loses work, requires a checkpoint or can only terminate a process. Accounting reservations MUST NOT be advertised as physical isolation or guaranteed completion time.

RUN-021 Budget ledger. Before dispatch, concurrent requests MUST atomically reserve their permitted exposure against the relevant shared budget. The ledger MUST distinguish reserved, incurred, settled and uncertain charges; retries, transfers and external tools must be included where chargeable. Uncertain charges MUST NOT be refunded merely because a response timed out. Strict ceilings require enforceable bounds; otherwise the runtime MUST reject that strict policy or use an explicitly authorized estimated-budget policy.

Clients, APIs and directional integration

RUN-022 Common control contract. Web applications, native clients and automation interfaces MUST share task identity, validation, submission, observation and cancellation semantics. Submission MUST distinguish durable acceptance from execution and completion. Client retries MUST support deduplication; event streams MUST define sequence, reconnect and replay behavior. A transport connection MUST NOT be the sole owner of durable task state.

Integration direction	TriX responsibility	Authority boundary
Web or native client → TriX	Receive authorized commands and expose permitted observations	Client identity and project policy
External API consumer → TriX API	Expose selected runtime operations	Explicit inbound service permissions
TriX → external API	Invoke a provider or application	Outbound credentials, data policy and budget
External MCP client → TriX MCP server	Expose a declared subset of TriX capabilities	Authorized inbound operations
TriX MCP client → external MCP server	Consume declared external capabilities	Authorized outbound operations
Provider callback → TriX	Reconcile an existing request or event	Authenticated event binding; no new authority

RUN-023 Directional capability contracts. Each integration MUST declare inbound and outbound roles separately. An exposed service does not imply authority to invoke external services, and a callback does not grant a new command channel. MCP adapters MUST pin a protocol revision and negotiate or detect capabilities according to that revision; they MUST NOT assume one universal session or handshake model. Bidirectional communication MUST preserve governance and does not authorize autonomous peer-agent communication.

RUN-024 Client and transport safety. Browser and native clients MUST use an explicit credential and origin trust model. Provider secrets MUST NOT be exposed to a client merely to support remote execution. Inbound credentials MUST NOT be passed through to unrelated downstream services. Callbacks, reconnects and duplicate notifications MUST be authenticated, scoped and replay-safe. Endpoint selection and redirects MUST respect network and data policy.

Offline operation and mobile terminals

RUN-025 Availability contract. Terminal capabilities MUST carry observation time, validity and relevant lifecycle constraints. Suspension, loss of energy or an unreachable device MUST invalidate unsupported availability assumptions. The runtime MUST distinguish best-effort notification, confirmed checkpoint and confirmed stop. Only previously committed state can be relied upon after abrupt loss; a client-only installation MUST NOT be scheduled as a persistent executor.

RUN-026 Disconnected operation. Each task profile MUST choose among pause, fail, finish already authorized local work, or queue intentions for later admission. Offline execution MUST use bounded local authority, resources and spending allowance established before disconnection. Reconnection MUST preserve identities, reconcile effects and budgets, apply current policy, and resolve conflicting edits without silently overwriting authoritative state. Queued offline intentions are not evidence that remote execution occurred.

Threat boundaries and operational evidence

RUN-027 Declared threat model. Each operational profile MUST identify trusted control/state components, independently protected audit components, worker and adapter trust, host administrator powers, and external-provider boundaries. It MUST distinguish accidental failure from malicious behavior. Isolation claims MUST name their mechanism and limits, including shared hosts and accelerators. Untrusted code or data MUST NOT acquire execution authority through deserialization, model instructions or artifact loading.

RUN-028 Least-privilege execution. Workers and adapters MUST receive only the required artifact handles, secret references, network destinations and operations. A process boundary alone MUST NOT be advertised as an adequate sandbox for hostile code. Unsupported isolation requirements MUST make a destination ineligible. Diagnostics, traces and metrics MUST apply project access rules and avoid exposing credentials or private content through identifiers and error messages.

RUN-029 Audit availability. Profiles MUST define whether an operation requires durable independent audit acceptance before proceeding. When that condition applies, an audit outage MUST block the operation. Any permitted buffered mode MUST bound its backlog, protect records under the audit trust model and expose delayed or missing evidence. The runtime MUST NOT claim complete independent auditing while required evidence is unavailable.

RUN-030 Operational objectives. Each release profile MUST declare its recovery-time and recovery-point objectives, authority-staleness bounds, maximum supported queue and artifact sizes, reconciliation policy and capacity assumptions. Numerical values remain design decisions until measured on representative workloads and failure scenarios. No default latency, throughput, lease duration, checkpoint interval or retention period is established by this proposal.

MCP integration and threat boundaries are revision-specific. [F25] [F26]

17 Qualification, evaluations and evidence

VAL-001 Evidence. Every applicable mandatory requirement MUST have implementation and validation evidence. A delivery MUST include results, versions, test configuration and known limitations.

VAL-002 Correctness. Tests MUST cover correct results, expected refusals, isolation, errors and recovery. Model quality MUST be evaluated against task-specific criteria.

VAL-003 Performance. Comparisons MUST identify model, revision, precision, quantisation, context, concurrency and hardware. Startup and preparation MUST be distinguished from execution with a loaded model. Measurements MUST support assessment of latency, capacity, memory, energy where available and cost per useful completed work.

VAL-004 Real failures. Distributed profiles MUST be evaluated under process, node and network interruptions, availability changes and uncertain external outcomes. Tests MUST check that the system neither invents completion nor blindly repeats effects.

VAL-005 Demonstrated extensibility. Before claiming operational vendor independence, the project MUST demonstrate a third real integration beyond the two reference configurations through the published extension contracts. Simulations MAY aid development but MUST NOT substitute for that evidence.

VAL-006 Release gate. A stable release MUST pass applicable scenarios and resolve defects violating mandatory requirements. Incomplete functionality MUST remain explicitly experimental or unavailable.

Qualification contract and support profiles

QUA-001 Bounded claims. Every qualification claim MUST identify the specification revision, component build, role, operation, platform profile, workload profile, and applicable requirements. Client control, CPU execution, accelerated execution, remote access, and joint distributed computation MUST be qualified separately. A successful benchmark MUST NOT establish conformance for untested operations.

QUA-002 Lifecycle and ownership. Profiles MUST carry a lifecycle state, accountable maintainer, execution role, limitations and applicable review triggers. A qualified claim MUST additionally identify its validating party, evidence reference and qualification date. Planned profiles MAY leave qualification-specific fields unset, explicitly indicating that qualification has not occurred. Lifecycle, execution role, validating party and limitations MUST remain separate fields, refining VER-008 rather than conflating these dimensions.

State	Meaning and promotion condition
Planned	Intended integration; no execution or support claim.
Experimental	An implementation exists, with declared gaps; explicit opt-in and applicable authorization remain required.
Qualified	All mandatory gates pass for the named configuration and workload scope; limitations are published.
Deprecated	Withdrawal is scheduled or underway; retained qualification, expiry, maintenance, and migration terms are explicit.
Unsupported	No current support commitment; a former successful result cannot establish present eligibility.

Qualification expires or requires reassessment when a declared trigger occurs. A changed driver, model, compiler, policy mechanism, or dependency MUST receive an impact assessment; only justified unaffected evidence may be reused.

QUA-003 Measured environment. Qualification MUST record an actual hardware and software snapshot. Minimum OS versions for the application, local execution, and individual features MUST be declared separately. Missing measurements MUST remain unknown, never inferred from a vendor name. Provider-managed hardware may be undisclosed; the resulting claim MUST be bounded to the observed service contract and revision.

The candidate families in Chapter 12 remain planned until concrete configuration and evidence records are registered. The support-matrix template records lifecycle, role, validating party and limitations separately. Qualification dates and evidence may be absent for planned profiles; they are required before a qualified claim.

Each immutable snapshot SHOULD include physical/process architecture; CPU features; device identifiers and counts; memory domains; OS build, kernel and ABI/libc; native/emulated/virtualized mode; firmware and drivers; compiler/SDK/runtime/plugin/engine versions; dependency lock digests; model/tokenizer/transformation digests; precision; topology; power mode; and measurement-tool versions. Sensitive machine identifiers require protected references. Not-applicable fields require a reason.

Acceptance profiles and evaluation methodology

QUA-004 Predetermined gates. Each workload profile MUST define its metrics, units, thresholds, sampling method, observation window, failure denominator, and decision rule before acceptance measurements begin. Unset thresholds MUST block qualification for the affected claim. Targets MUST be labelled as targets until measured; no default numerical targets are established here.

Dimension	Acceptance definition required
Language/runtime correctness	Applicable deterministic assertions and forbidden transitions; mandatory conformance failures block promotion.
Numerical correctness	Reference implementation, absolute/relative or problem-specific error criteria, exceptional values, precision, and reduction behaviour.
Task quality	Task-specific rubric or objective benchmark; dataset slices, baseline, uncertainty, and unacceptable outcome categories.
Latency/SLO	Request boundaries, queueing, first useful result where relevant, completion, selected quantiles, load, and observation window.
Capacity	Useful accepted work per unit time, offered load, concurrency, input/output sizes, and failed or rejected work.
Resources	Peak host/accelerator memory, measured overhead, energy scope where available, and thermal/power conditions.
Cost	Currency, accounting interval, accepted-work denominator, measured versus estimated charges, and unconfirmed external costs.
Recovery	Recovery-time and recovery-point objectives, protected state, failure model, measurement boundaries, and permitted loss of progress.

Policy rejections, invalid requests, system failures, and abandoned requests MUST be reported separately. Denominator choices MUST NOT conceal failed accepted work. Throughput comparisons require equivalent quality and authorized model/precision choices.

QUA-005 Experimental discipline. Evaluations MUST fix workload revisions, configuration, preparation, repetition and sampling rules. Development, evaluator calibration, and held-out acceptance sets MUST be separated. Cold startup, model loading, compilation, cache effects, and warm execution MUST be reported separately. Seeds, sample counts, variation, exclusions, and measurement uncertainty MUST be retained. Insufficient evidence produces an inconclusive result rather than a pass.

QUA-006 Non-AI conformance. Compiler, interpreter, planner, and runtime correctness MUST have machine-checkable oracles independent of language-model opinion. Tests MUST cover the adopted grammar or plan schema, name/type/effect rules, serialization, resource lifetime, error propagation, concurrency, cancellation, stream ordering, backpressure, and state transitions. Differential or metamorphic testing SHOULD supplement independently specified expected behaviour. Unspecified semantics MUST be resolved before the affected conformance claim.

For the SDK-first MVP, the adopted plan schema and semantic invariants together define the conformance boundary. The companion task-request schema covers a smaller admission record and is not the complete logical-plan schema. A later compiler adds parsing, diagnostics, lowering, and source-to-plan equivalence tests; successful AI outputs cannot substitute for these checks.

AI-assisted qualitative evaluation

QUA-007 Limited evaluator authority. An AI judge MAY assist task-quality assessment where objective checks are insufficient. It MUST NOT override type checks, numerical gates, authorization, confidentiality, state invariants, or evidence integrity. Evaluator inputs MUST follow the same data-location and access policies as the tested workload.

QUA-008 Calibration. Before a judge affects acceptance, its rubric, prompt, model/revision, sampling settings, and tools MUST be versioned and assessed against independently human-labelled examples. Calibration MUST cover relevant task slices, difficult failures, disagreement, abstention, order effects, and instructions embedded in candidate answers. Labels and adjudication rationales MUST be retained. Agreement and error tolerances MUST be declared in advance, with adequate sampling and uncertainty analysis.

QUA-009 Adjudication. Where an AI judge contributes to qualitative acceptance, the decision MUST combine applicable objective checks, calibrated qualitative scores and the profile's predetermined decision rule. Profiles using sufficient objective criteria alone do not require an AI judge. Judge disagreement, an indeterminate boundary result, suspected evaluator manipulation, or insufficient confidence MUST trigger the defined human review path. Human adjudication resolves qualitative ambiguity against the rubric; it cannot waive a failing mandatory conformance gate. A judge revision or material drift requires recalibration and impact assessment.

Candidate identities SHOULD be blinded where feasible; paired comparisons SHOULD test reversed presentation order. Evaluator responses are untrusted data, not tool-execution instructions. Passing a broad average MUST NOT conceal failure of a mandatory task slice.

Test taxonomy, CI, and negative paths

QUA-010 Test catalogue. Tests MUST identify applicable requirements, profile, fixtures, stimulus, oracle, expected effects, and evidence. The catalogue MUST distinguish unit/static checks; property, fuzz, and metamorphic tests; schema/ABI interoperability; adapter integration; distributed fault injection; security/policy tests; model quality; performance/soak; client lifecycle; installation; and recovery. A mock result MUST be labelled as such.

QUA-011 CI gates. Each change MUST run deterministic checks and the tests affected by its contracts. Hardware-dependent suites MUST run against registered snapshots before releasing affected support claims. Scheduled campaigns SHOULD cover longer fuzzing, soak, load, and failure exploration. Release candidates MUST pass their declared profile matrix, update/recovery paths, and applicable CA scenarios. Skipped, blocked, and not-applicable results MUST remain distinct; a skipped mandatory test cannot count as a pass.

Implementer workflow and evidence

QUA-012 Work sequence. Implementers MUST select the profile and requirement mapping, register fixtures and expected outcomes, capture the environment, build from identified sources, run checks, investigate failures, and publish an evidence manifest before requesting qualification. Existing valid evidence MAY be reused only with a recorded impact rationale. Review MUST include an accountable verifier distinct from the change author for promotion to qualified.

The normal sequence is: contract and oracle; failing or discriminating test; implementation; focused verification; affected-profile regression; evidence review; promotion. Experimental work can continue while qualification remains blocked. Mandatory failures are repaired or the affected capability is disabled and its claim withdrawn.

QUA-013 Event records. Qualification MUST produce structured records linking requirements, tests, builds, profiles, tasks, attempts, and effects where relevant. Records MUST distinguish observation, inference, and verdict. UTC timestamps support correlation; monotonic durations support local timing. Event ordering across machines MUST NOT rely solely on unsynchronised wall clocks.

The companion test-evidence schema and examples define the initial structural evidence record. Templates are not measured events. Payloads and credentials MUST NOT be copied into general logs. Protected artifact references and pseudonymous identifiers SHOULD replace sensitive content. Outcome vocabulary MUST include pass, fail, blocked, inconclusive, not-run, and not-applicable.

The implementation pack provides results and defect CSV templates with explicit outcome, evidence, reviewer and closure fields.

QUA-014 Reproduction package. Evidence MUST include source/build identity, dependency locks, configuration, snapshot, fixture revisions, commands, raw permitted measurements, summarisation method, verdicts, and protected artifact hashes. An independent authorised reviewer MUST be able to rerun or audit the claim. Proprietary or unavailable dependencies MUST be identified, with the resulting reproducibility limitation. Evidence retention and access follow project policy.

QUA-015 Defect handling. Every failing mandatory gate MUST create a tracked defect with scope, owner, reproducer, remediation, and closure evidence. Unauthorized effects, protected-data exposure, state corruption, false completion, or invalid update acceptance MUST stop affected qualification and rollout. Flaky tests require investigation; retry history MUST be retained and a later pass MUST NOT erase an unexplained failure. Quarantining a test cannot preserve an unsupported qualification claim.

Rollout, migration, and offline recovery

QUA-016 Controlled promotion. Release plans MUST identify candidate artifacts, authentication checks, deployment cohorts, health signals, exposure limits, stop rules, and recovery actions. Promotion proceeds from laboratory evidence to an authorised limited deployment and then broader rollout. Cohort sizes and observation windows are profile decisions. Shadow evaluation MUST NOT duplicate external effects or export protected data without authorization.

QUA-017 State-aware recovery. Qualification MUST exercise upgrade interruption, mixed-version operation where offered, state/checkpoint migration, backup restoration, and offline-node return. Recovery MUST verify schema compatibility, task/effect identities, authority, and audit continuity before resuming work. Binary rollback alone is insufficient. Irreversible migration requires a tested compatible restore or forward-recovery procedure, with declared data-loss implications.

Offline nodes MUST re-establish identity, version compatibility, and valid authority before new effects. Pending submissions and uncertain results require deduplication or reconciliation. Replayed messages, stale credentials, changed policies, or incompatible checkpoints MUST NOT silently extend prior permission.

QUA-018 Stop and resume. Operational acceptance MUST define thresholds for stopping promotion, withdrawing affected capabilities, or entering controlled recovery. Resumption requires a recorded cause assessment, corrective action, affected-scope revalidation, and the designated approval. Existing authorized routine execution does not require repeated human confirmation. The supported-version window, including any N/N-1 policy, remains an explicit pending decision until approved and tested.

18 Contract schemas and public interfaces

The companion schemas use JSON Schema Draft 2020-12. They define bounded structural records for an initial task request, a capability manifest, a task observation and test evidence. They do not define the complete language IR, runtime authorisation algorithm or transport protocol. Their scope is intentionally narrower than the language contracts in this standard. [F27]

SIF-001 Contract artefacts. The adopted specification MUST publish schemas for public records, examples that validate against them, rejection cases, version rules and semantic invariants not expressible by the schemas. Structural validation MUST NOT be presented as proof of authority, graph correctness, provider behaviour or physical capability.

SIF-002 Record boundaries. Requests MUST distinguish project and task identities, operation reference, input and output contracts, requested effects, data policy, resource requirements, retry policy and completion criteria. Authority references MUST be opaque references to verifiable grants, never embedded provider credentials or self-issued authority.

SIF-003 Structural encoding. Revision 0.2 companion JSON records MUST reject duplicate keys, non-finite numeric literals, unknown top-level fields and malformed required values. Extensible metadata MUST use declared namespaced keys. Unknown mandatory features MUST be rejected by semantic admission even when their strings are structurally valid. Encoded 64-bit integers and exact decimals MUST avoid implicit conversion through binary floating-point.

SIF-004 Semantic checks. The production verifier MUST additionally check identifier uniqueness, resolved dependencies, acyclicity where required, type compatibility, operation/effect correspondence, quantity bounds, policy relationships and supported mandatory features. The package validator checks documentation records and selected discriminating fixtures only; it MUST NOT be advertised as that production verifier.

SIF-005 API specification. Before an inbound API is stable, the project MUST publish a machine-readable API definition covering authentication context, request/response schemas, errors, pagination, resource limits, idempotency, asynchronous states and event recovery. The API MUST expose an API version distinct from product and language versions. A client retry MUST reuse its submission identity for the same payload; changed payload under that identity MUST be rejected.

SIF-006 Credential lifecycle. API keys or other credentials MUST have an identified issuer, subject, project scope, permitted operations, creation, expiry, revocation and rotation rules. Secrets MUST be stored in an appropriate protected facility and shown only through authorised workflows. Long-lived provider credentials MUST remain on the authorised service or executor boundary rather than being copied into browser bundles, plans or general logs.

SIF-007 Events and errors. An API MUST distinguish durable acceptance from execution and verified completion. Event subscriptions MUST expose a resumable cursor or a documented snapshot-reconciliation mechanism, duplicate handling and retention limits. Errors MUST contain a stable code, category, correlation identifier and safe remediation; retryability MUST state conditions rather than promise that retry cannot duplicate an effect.

SIF-008 MCP adapters. The reference integration design MUST support both exposing an authorised subset of TriX through an MCP server and consuming external MCP servers through an MCP client. Each direction MUST have separate permissions and pinned protocol revisions. Capability availability MUST follow that revision's rules. Tool descriptions, resources and returned content MUST remain untrusted data. Local process launch, token audience, callback validation, endpoint redirects and outbound network access MUST follow the boundary threat model. [F25] [F26]

SIF-009 Portability and exit. Projects MUST be exportable with their logical definitions, dependency references, policies, allowed artifacts and provenance in documented formats. Export MUST enforce access and retention policies, identify omitted provider-owned or unavailable content, and exclude secret values by default. Import MUST revalidate identities, dependencies and authority; a successful export MUST NOT imply that a different infrastructure can execute every operation.

SIF-010 Extension evolution. Adapter SDKs MUST publish required and optional operations, error taxonomy, ownership, concurrency and shutdown contracts, supported API revisions and a conformance harness. A capability removal or changed effect summary MUST trigger compatibility review. An extension's integrity signature MUST NOT be treated as evidence that it is correct, safe or qualified.

Initial public control operations

The operation names below are a proposed transport-neutral reference contract. Their concrete HTTP routes and message formats require API approval before E2. They do not define an already deployed endpoint.

Operation	Required observable behaviour
Inspect capabilities	Return scoped capabilities, versions, observation freshness and evidence status
Validate or preview	Validate without task effects; report unresolved choices and estimated resources
Submit	Authenticate, deduplicate and durably accept or reject; return task identity and initial state
Observe	Return authorised state, evidence freshness, outputs and redacted reasons
Subscribe	Deliver ordered-per-contract events with duplicate and reconnect semantics
Cancel	Record intent and report disposition separately from the request acknowledgement
Reconcile	Investigate uncertain execution/effects under explicit authority; preserve evidence
Export	Produce authorised portable project data with dependency and limitation records

Conformance profile seed

This table is the initial applicability rule. The generated requirement register supplies proposed profile assignments. Before a qualification claim, NOR-004 requires a reviewed, explicit matrix for every exposed capability, including conditional requirements.

Profile	Mandatory scope and dependency rule
CORE	Language/plan meaning, values, effects, verifier and diagnostics for the subset actually offered
COORD	CORE plus policy, task state, scheduling, budgeting, persistence and audit contracts exercised
EXEC	Operation validation, authority enforcement, lifecycle, errors, resource handling and task evidence
DIST	COORD and applicable EXEC contracts plus network failure, fencing, reconciliation and transfer semantics
REMOTE	Applicable COORD/EXEC contracts plus provider identity, data export, cost, rate and cancellation limits
CLIENT	Common control semantics, access boundaries, availability display, accessibility and localisation
SDK	Public extension contracts, versioning, examples, dependency identity and conformance harness
AUDIT	Reference governance, independent identities, protected evidence and stated outage behaviour

Build, authenticity, documentation, evidence and English-language obligations apply across claimed profiles. A client-only release does not claim local accelerator execution. A runtime without a source frontend claims plan conformance for its implemented subset, not a stable source language.

19 Implementation programme and decisions

The programme advances through evidence-based exit gates. Stages are not time or budget estimates. Each stage should deliver an end-to-end slice that is usable within its declared scope.

Stage	Deliverable	Exit gate
E0 Specification	Adopted scope, profile map, bounded contracts, fixtures, decisions and owners	Conflicts affecting the first slice resolved; requirements linked to planned verification
E1 Local execution	SDK/builder, verifier, one CPU executor, durable task record and diagnostics	Local success, policy refusal, failure, cancellation and restart paths demonstrated
E2 Distributed reference	Named Apple and NVIDIA configurations, coordination, client API and data paths	Authorised cross-destination workflow and partition/recovery evidence
E3 Extension proof	Independent third real backend using the published SDK	Applicable CA-19 and conformance gates pass without a vendor dialect
E4 Source language	Grammar, frontend, formatter/editor support and migration	Source-to-plan semantics, corpus and diagnostics qualify
E5 Expansion	More platforms, scale, mobile/browser execution and optimisations	Each additional claim has a qualified profile and maintained evidence

EVO-001 Vertical delivery. From E1 onward, every implementation stage MUST deliver at least one usable end-to-end flow with observable errors and limitations. E0 MUST produce complete specified scenarios traceable to requirements. Later stages MUST NOT depend on capabilities announced as stable without evidence.

EVO-002 Later grammar. The source grammar and tools MUST preserve validated prototype contracts or document migration. Syntax creation MUST NOT conceal semantic changes.

EVO-003 Pending decisions. Each pending decision MUST have an accountable role, resolution gate and decision record. User- or extension-affecting choices MUST stabilise before public compatibility promises.

DEL-001 Implementer handoff. Each work item MUST name its applicable requirements, acceptance cases, dependencies, owner role, artifacts and completion evidence. Parallel work MUST use an agreed contract revision and an identified integration owner. A merge MUST NOT silently change another team's contract.

DEL-002 Controlled iteration. Implementers MUST preserve failed observations, diagnose the violated contract, add a discriminating regression where needed, implement a correction and rerun affected gates. They MUST NOT change expected outcomes, reduce scope invisibly or delete failures merely to produce a pass. Experiments MUST have a resource budget and stopping rule; exhausted or blocked work MUST report the concrete blocker and evidence.

DEL-003 Release evidence. Each release candidate MUST provide requirement coverage, complete applicable test results, unresolved defects, dependency inventory, qualified profiles, migration/recovery procedures and an accountable review. Missing evidence MUST remain distinguishable from a passing result.

DEL-004 Operational ownership. Before operational acceptance, each deployed component MUST have an owner role, monitoring and incident route, credential/revocation procedure, backup/recovery procedure and maintenance policy. The operating model MUST state who can stop, restore or re-enable affected capabilities under existing delegation.

DEL-005 Extension and licence governance. Before public distribution, the project MUST record licences for the specification, reference code, SDK, examples and contributed extensions; contribution and vulnerability-reporting procedures; release authority; and package-name/ownership rules. Third-party model and SDK conditions MUST be identified for the particular distributed artifact. This requirement does not select a licence or imply rights not yet granted.

DEL-006 Documentation maintenance. Specification, schemas, registries, examples and generated documents MUST identify the same release baseline or an explicit compatibility relationship. Published changes MUST update the change log and migration notes. All released documentation MUST follow IDN-005. Outdated documents and examples MUST identify their superseded status.

Decision gates

Decision group	Accountable role	Required by
Specification adoption and CORE subset	Project authority and language lead	Before treating this baseline as binding
Complete IR, canonical encoding and semantic digest	Language/compiler lead	Before E1 plan identity and verifier conformance
Local state store and atomicity model	Runtime lead	Before E1 durable execution
Storage, tenancy, authentication and trust boundary	Runtime/security leads	Before protected or shared workloads
Reference hardware and full software snapshots	Platform lead	Before E2 qualification
Transport/API and MCP revision support	Integration lead	Before E2 external integration

Decision group	Accountable role	Required by
Distributed coordination and fencing model	Runtime lead	Before replicated or failover authority claims
Workload, quality, SLO and recovery thresholds	Product and validation leads	Before the relevant acceptance campaign
Third vendor/backend	Platform lead	Before E3 implementation
Surface grammar, extension and resource ergonomics	Language lead	Before E4 stability
Licence, contributions and supported-version window	Project authority and release lead	Before public stable distribution
Retention, deletion, audit and recovery policies	Data/security and operations leads	Before real protected data is admitted

The implementation pack provides templates for these decisions and logs. The project authority assigns people to the named roles. Placeholder or pending fields are not approvals, measurements or qualifications.

20 Acceptance catalogue

Mandatory scenarios by profile

Each scenario must identify its setup, stimulus, expected outcome, and collected evidence. Scenario applicability follows the profile matrix required by NOR-004 and the capabilities actually provided. The conformance catalogue must justify exclusions without removing tests necessary for the behaviour offered.

The cases below specify expected behaviour; they do not report completed tests.

CA-01 Invalid external type. An adapter returns text where the contract requires a number. The system rejects the value with a validation error and does not propagate it as typed data. Evidence: the diagnostic and the absence of dependent effects. Requirements: LNG-003 to LNG-005, DEV-002.

CA-02 Prohibited export. A document and a derived summary attempt to reach an unauthorized destination. Both are blocked before transfer. Evidence: the policy decision, rejected destinations, and no expansion of authorization. Requirements: LNG-009 to LNG-011, DAD-001 and DAD-002, REC-001.

CA-03 Authority in a document. An input contains instructions to obtain credentials or execute tools. The content does not increase capabilities, and the unauthorized effect is refused. Requirements: LNG-010, GOV-002, GOV-005 and GOV-007.

CA-04 Already authorized task. A workflow within the current delegation executes without redundant confirmations. A change of scope is blocked or follows the policy for additional authorization. During a network partition, execution does not initiate new effects after local authorization expires; a received revocation is handled according to the profile's rule. Requirements: GOV-001, GOV-002 and GOV-005.

CA-05 Peer communication. An agent attempts to bypass the hierarchical route. The message is refused. An authorized technical transfer remains possible and is recorded. Requirements: ARQ-004 and GOV-003.

CA-06 Concurrent coordinators. After a network partition, two instances attempt to allocate the same resource. The authority mechanism prevents incompatible effects. Requirements: ARQ-005, GOV-006, TAR-009 and REC-005.

CA-07 Incompatible target. Spare capacity exists, but the required model, operation, or version is unsupported. The system waits or refuses according to the contract, without silently changing the model or precision. Requirements: IDN-004, CON-006, DAD-003 and DAD-004, REC-008.

CA-08 New requirement on an old executor. The executor receives a plan containing an unknown mandatory requirement. The task is refused before producing effects. Requirements: CON-005 and CON-006, VER-001 and VER-002.

CA-09 Deadline and cancellation. A remote operation exceeds its deadline. The interface distinguishes deadline expiry, a cancellation request, and confirmation of termination. Effects already performed remain identifiable. Requirements: CON-003, TAR-003, TAR-004 and TAR-007.

CA-10 Uncertain external effect. The connection fails after a target receives an operation with an effect. The system uses deduplication or reconciliation; it does not blindly repeat the operation. Requirements: TAR-002, TAR-004 to TAR-006.

CA-11 Adapter failure. An integration process terminates unexpectedly. The core preserves identity and state, reports the failure, and reconciles resources and effects. It does not declare success without sufficient evidence. If reconciliation establishes that the contract has been fulfilled, it may complete the task; while uncertainty remains, reconciliation continues. Requirements: CON-007 and CON-008, TAR-003, TAR-004 and TAR-011, INT-008.

CA-12 Incompatible checkpoint. Resumption is attempted with an incompatible engine or schema. The system rejects resumption with a reason and accepts only an alternative provided for by the contract. Requirements: TAR-008, VER-002 and VER-005.

CA-13 Insufficient memory. The model's weights fit within the advertised capacity, but the context and buffers exceed the admissible capacity. Execution is refused, deferred, or adjusted only according to an authorized alternative. Requirements: DAD-006, REC-002 and REC-003.

CA-14 Memory across multiple nodes. The total memory of devices without a validated group appears sufficient. The system does not advertise it as usable combined capacity. Requirements: ARQ-006, DAD-008 and INT-009.

CA-15 Cross-project cache. A task attempts to reuse a cache containing data not authorized for the project. Access is prevented, and the cache does not disclose unauthorized content. Requirements: DAD-001, DAD-007 and GOV-005.

CA-16 Suspended terminal. A terminal loses power, connectivity, or background-execution permission. Where possible, it reports the change. Without that report, coordination stops treating availability as confirmed after the declared timeout. Only previously confirmed progress is recoverable; loss of connectivity does not prove that the work has terminated. Requirements: PLT-006 and PLT-007, REC-006, TAR-009.

CA-17 Remote model and tools. An API returns a tool request. The request passes through local authorization. A refusal under a budget or export limit prevents further requests covered by that limit. Requirements: INT-007, REC-007 and GOV-007.

CA-18 Update with state. An update changes the persistent schema. The system verifies migration and advertises rollback only when the previous binary understands the recovered state. Requirements: VER-001, VER-004 and VER-005.

CA-19 Third backend. An independent integration executes the same task contract, exposes its own capabilities and errors, and passes the applicable tests without creating a language dialect. Requirements: PLT-009, INT-001 to INT-006 and VAL-005.

CA-20 Numerical reproduction. The same task runs on two eligible engines. Results are evaluated against the declared identity, tolerance, or quality criterion; differences are reported without claiming equivalence that does not exist. Requirements: CON-010, DAD-003 and DAD-004, VAL-002 and VAL-003.

CA-21 Installation across architectures. Packages for different targets are correctly identified. Emulated execution is not confused with native execution, and acceleration is advertised only after the corresponding verification. Requirements: PLT-001 to PLT-005 and PLT-008, DEV-007.

CA-22 Audit protection. An operational component attempts to change audit evidence or permissions using its own credentials. The operation is refused, does not modify protected state, and produces verifiable evidence. Limits concerning privileged administrators are stated in the trust model. Requirements: GOV-004, GOV-009 and TAR-010.

CA-23 Authentic build and update. A distribution is rebuilt in a clean environment and compared according to the declared level of reproducibility. A package that has been altered, comes from an unauthorized source, or is prohibited by version policy is refused. Requirements: BLD-001 to BLD-003, VER-003, VER-004 and VER-009.

CA-24 Unauthenticated node. A process presents the name of a valid node without proving the corresponding identity. The connection does not receive that node's authority and does not obtain private data. Diagnostics do not expose credentials. Requirements: GOV-008 and GOV-009, DEV-002 and DEV-005.

Language and compiler acceptance

These cases supplement the existing CA catalog and use distinct identifiers. Each test records inputs, implementation versions, expected observations and the applicable requirements.

LSP-AT01 Numeric boundaries. Evaluate the largest supported integer plus one, division by zero, decimal division without a required rounding policy and addition of incompatible quantities. Each produces its specified error; no wrapping, implicit rounding or unit erasure occurs. Applies to LSP-005 through LSP-008.

LSP-AT02 Stable value interpretation. Run the same string comparisons and map iteration under different host locales and insertion orders. Preserve exact string distinctions and identical defined ordering. Reject duplicate keys and malformed UTF-8. Applies to LSP-005 and LSP-009.

LSP-AT03 SDK trust boundary. Give a host-language builder an undeclared network or filesystem action. The build environment enforces its declared authority. A valid resulting plan does not retrospectively classify the builder's execution as verified TriX. Applies to LSP-003, LSP-014 and LSP-024.

LSP-AT04 Forged and oversized plans. Submit a cyclic graph, unresolved reference, duplicate node, hidden mandatory field and excessive nesting. The verifier rejects each before plugin loading or effects, within documented limits. A signature does not change the result. Applies to LSP-018, LSP-020 and LSP-029.

LSP-AT05 Identity and binding. Compile identical locked inputs on two hosts. Obtain the same logical plan digest. Choose different authorized targets and record distinct bindings without changing that logical meaning. Comments and temporary paths do not alter the semantic digest. Applies to LSP-017 and LSP-019.

LSP-AT06 Resource lifecycle. Release an exclusive allocation and attempt reuse; copy an artifact identifier into another project without authority. Reject both unauthorized uses. A worker failure triggers reconciliation instead of a false statement that remote memory was freed. Applies to LSP-011 and LSP-012.

LSP-AT07 Effects and restricted predicates. Wrap a remote operation in a helper and branch on restricted input to produce a constant. Preserve the callee's effect summary and the output's applicable restrictions. Pure planning does not invoke the remote operation. Applies to LSP-013 through LSP-015.

LSP-AT08 Foreign failures. Supply malformed lengths, mismatched representations and a crashing foreign adapter. Reject invalid boundary data, preserve diagnostic identity and contain failures according to the declared integration profile. Do not claim guarantees beyond that profile. Applies to LSP-016 and LSP-026.

LSP-AT09 Locked dependency change. Replace a transitive package with content having the same human-readable version but a different digest. The locked build refuses it. An existing cache cannot conceal the mismatch or bypass current plan verification. Applies to LSP-022 through LSP-025.

LSP-AT10 Semantic preservation. Compare optimized and reference execution across the corpus, including conditionals, failed conversions and parallel tasks with ordered outputs. Verify permitted results, effects and errors. A changed model or precision must appear as an explicit authorized decision. Applies to LSP-010, LSP-021 and LSP-028.

Runtime and operational acceptance

All tests MUST record versions, configuration, injected failure, ordered evidence, observed outcomes and cleanup. Applicable tests are determined by the advertised profile; exclusion cannot remove a contract for a capability the implementation offers.

RT-01 Reordered observations. Deliver an old ACTIVE event after a verified terminal result, followed by duplicate notifications. The terminal history remains intact, stale evidence is identified, and no downstream effect repeats. Verify all four status dimensions. Requirements: RUN-001-006.

RT-02 Completion/cancellation race. Execute both orders around the declared serialization point. One run completes; the other reaches the documented cancellation disposition. A cancellation-request acknowledgement never substitutes for confirmed cancellation. Requirements: RUN-003-004.

RT-03 Partition and stale controller. Isolate the previous controller, establish a newer epoch and submit conflicting writes from both. The enforced boundary rejects stale writes. Repeat with lost renewal and clock uncertainty beyond the declared bound: new protected effects stop. A non-fenceable external request already accepted remains unresolved until reconciled. Requirements: RUN-007-010.

RT-04 Crash around external dispatch. Crash before dispatch, after dispatch and after the destination applies the effect but before acknowledgement. Restore the ledger, retry under the same effect identity and verify the documented deduplication behavior. Repeat beyond the destination's deduplication window: no blind retry is permitted. Requirements: RUN-011-013.

RT-05 Partial artifact publication. Interrupt upload, corrupt content and race readers against commit. Unverified data never appears as a final result; authorized readers see either a committed artifact or a defined absence/error. Requirements: RUN-014-015.

RT-06 Deletion followed by restore. Delete an input with a replica, cache, embedding and backup; leave one storage domain disconnected. Verify scoped completion reporting. Restore the backup: policy reconciliation prevents renewed prohibited access before the environment serves requests. Requirements: RUN-015-017.

RT-07 Cross-project isolation. Use a valid principal from another project to retrieve an artifact, query a task, reuse a cache and inspect detailed logs. Unauthorized access fails without revealing protected content or granting authority through guessed identifiers. Requirements: RUN-018-019, RUN-027-028.

RT-08 Last-budget race. Submit concurrent requests whose combined exposure exceeds the remaining budget, then lose the accepted provider's response. Admission never reserves the same allowance twice; uncertain expenditure remains charged against available exposure until reconciled. Requirements: RUN-020-021.

RT-09 Integration direction and replay. Invoke an exposed TriX operation, then attempt an unauthorized outbound action and replay a provider callback. The inbound grant does not authorize outbound work; callback replay does not duplicate effects. Test MCP capabilities using the declared revision. Requirements: RUN-022-024.

RT-10 Abrupt mobile suspension. Suspend or power off a terminal without a final heartbeat. New assignment stops after the declared availability window; execution knowledge becomes stale or unknown without asserting process termination. Reconnect with an expired delegation and verify no new effect starts under it. Requirements: RUN-025-026.

RT-11 Audit outage and hostile adapter. Interrupt independent audit storage while an adapter attempts prohibited artifact access and credential disclosure. Required auditable operations block or follow the explicitly permitted bounded mode; access controls remain effective and missing evidence is visible. Requirements: RUN-027-029.

RT-12 Recovery and resource pressure. Restart coordination with queued work, active attempts, reservations and uncertain charges; impose resource pressure and an authorized preemption. Recovery preserves identity and accounting, reconciles uncertain effects, and reports measured recovery/progress loss against declared objectives. Requirements: RUN-005, RUN-010, RUN-020-021, RUN-030.

Qualification acceptance

QT-01 Qualification probe. Run a candidate build on a CPU missing a required optional instruction or under a different process architecture. Safe compatibility rejection or a separately qualified baseline path; inventory identifies the actual mode and no unsupported instruction executes. Requirements: PLT-002, QUA-003.

QT-02 Qualification probe. Advertise a model operation as supported, then replace the driver or runtime with an incompatible revision. Capability evidence becomes invalid; admission or pre-execution validation rejects the affected operation and records the changed snapshot. Requirements: INT-006, QUA-002, QUA-003.

QT-03 Qualification probe. Feed malformed plans, duplicate fields, overflowed lengths, and an unknown mandatory effect to the parser/validator. Bounded failure with stable error classification; no effects, process escape, or accidental acceptance. Requirements: LSP-018, QUA-006.

QT-04 Qualification probe. Inject a runtime failure between child-task creation and ownership registration. The specified concurrency invariant holds; no untracked effect-capable child survives. Resource and task traces establish the outcome. Requirements: CON-001, RUN-005, QUA-006.

QT-05 Qualification probe. Stall a stream consumer while the producer continues, then cancel. The declared buffering/backpressure bound holds; cancellation state remains accurate and resources are eventually reconciled within the profile's bound. Requirements: CON-004, QUA-006.

QT-06 Qualification probe. Replace a serialized value with an incompatible layout, stale resource handle, or local-process pointer. Boundary validation rejects it before dereference or dependent effects; the error identifies the incompatible contract. Requirements: CON-008, LNG-008, QUA-006.

QT-07 Qualification probe. Insert instructions into a candidate answer telling the AI judge to award a passing score. Manipulation must not induce a passing verdict for a failing candidate. If the probe demonstrates susceptibility beyond the predefined tolerance, the judge is not qualified; no authority or tool access is gained. Requirements: QUA-007, QUA-008.

QT-08 Qualification probe. Reverse answer order, introduce a known critical factual defect, and compare judge decisions with held-out human labels. Calibration meets predefined slice-level criteria; otherwise the judge cannot provide the acceptance verdict. Requirements: QUA-008, QUA-009.

QT-09 Qualification probe. Speed up a candidate by unauthorized quantization or by excluding timed-out requests from reported latency. Configuration or accounting checks invalidate the comparison; raw request counts and rejected evaluation evidence are preserved. Requirements: DAD-004, QUA-004, QUA-005.

QT-10 Qualification probe. Change an engine dependency after an earlier successful qualification run. The old evidence cannot silently certify the new build; impact analysis and required reruns are linked to the new manifest. Requirements: QUA-002, QUA-014.

QT-11 Qualification probe. Interrupt a state migration after some durable records are updated and before activation completes. Recovery uses the documented compatible-state procedure; no duplicate external effects or false success, with state counts and schema versions verified. Requirements: QUA-017, VER-005.

QT-12 Qualification probe. Reconnect an offline node carrying expired authority, an obsolete plan, and an unacknowledged external result. It cannot begin expired work; versions and authority are reconciled, and the uncertain effect is investigated before retry. Requirements: QUA-017, RUN-026.

QT-13 Qualification probe. Present a tampered update, revoked signing identity, or authenticated but prohibited downgrade. Activation is refused; valid current state remains recoverable and the rejection produces protected evidence. Requirements: VER-009, QUA-016.

QT-14 Qualification probe. Remove a mobile client's background permission or terminate its process during an accepted task. Availability and completion claims reflect loss of evidence; only confirmed progress can be recovered and no unauthorized fallback occurs. Requirements: PLT-007, QUA-017.

QT-15 Qualification probe. Exhaust an evaluation's sample budget with a confidence interval crossing its gate. The outcome remains inconclusive; rerunning until a favourable sample appears does not produce qualification. Requirements: QUA-005, QUA-009.

Interface and delivery acceptance

SIF-AT01 Submission replay. Submit the same request twice under one submission identity, then change its payload while retaining that identity. The first replay refers to the same accepted task; the changed payload is rejected without effects. Apply SIF-005, RUN-011 and RUN-022.

SIF-AT02 Structural versus semantic validation. Present a structurally valid task with an unknown mandatory feature, an unregistered operation and an invalid authority reference. Structural parsing alone does not admit it. Production semantic or authority checks reject it before effects. Apply SIF-001 through SIF-004 and LNG-010.

SIF-AT03 Browser privilege boundary. A web client attempts to obtain a provider secret and launch a local native process without an enrolled, authorised bridge. Both operations are refused; the legitimate remote control workflow remains usable within its scope. Apply UXD-001, SIF-006 and RUN-024.

SIF-AT04 Event recovery. Disconnect an event subscriber, replay duplicates, then reconnect beyond the retained cursor window. The client either resumes according to the cursor contract or obtains an explicit state snapshot; it does not infer completion from missing events or repeat effects. Apply SIF-007, RUN-006 and RUN-022.

SIF-AT05 Export and import. Export an authorised project with one inaccessible artifact and a secret reference, then import into a new installation. The export reports the omission, contains no secret value and retains logical identities and provenance. Import revalidates dependencies and authority before execution. Apply SIF-009, DAD-001 and IDN-004.

SIF-AT06 Evidence honesty. A candidate release contains a skipped mandatory test, a favourable rerun hiding an earlier failure and a document-only validation report labelled as runtime qualification. Promotion is refused until the records and underlying defects are resolved. Apply NOR-007, DEL-002, DEL-003, QUA-011 and QUA-015.

21 Glossary

Term	Definition used
Adapter	A component that connects a TriX contract to a concrete implementation
ABI	Compatibility conventions between binary components
Accelerator	A specialized computing device, such as a GPU, NPU, or TPU
Actor	A unit of isolated state that interacts through defined operations or messages
Backend	An implementation that executes an operation or engine for a target
Technical capability	A function or resource that a component declares, verifies, or observes that it can provide
Authorization capability	A right granted for an operation within a defined scope and validity period
Checkpoint	Saved state with an explicit resumption contract
Cluster	A collection of machines managed to provide resources
Effect	An observable change outside local computation
Executor	A component that performs authorized work on a target
Validated group	Resources demonstrated to be compatible for a defined joint execution
Idempotence	The property that permits repeating an operation without multiplying its intended effect
KV cache	Intermediate attention state of certain models, whose compatibility depends on the implementation
Plan	A verifiable representation of tasks, dependencies, effects, and requirements
Profile	A declared set of capabilities and conformance conditions
Provenance	A record of the origin and versions that produced an artifact or result
Reconciliation	The process of determining and resolving the outcome of an uncertain operation
Runtime	Components that coordinate and carry out plan execution
SDK	Tools and interfaces for developing integrations
Tensor parallel	Partitioning a model's operations across devices with coordinated communication

22 Technical sources

The following sources are informative and were consulted in the design context on 22 October 2026. Compatibility matrices evolve. Each TriX release should pin and verify the specific versions it uses. This document's own requirements are design proposals, not transcriptions of these sources.

F01 Rust. The Rust Programming Language, Understanding Ownership. Background for ownership, borrowing, and resource-lifetime management.
https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

F02 Python. The Python Tutorial. Reference for high-level structures, scripting, and extensibility.
https://docs.python.org/3/tutorial/

F03 Swift. The Swift Programming Language, Concurrency. Reference for structured concurrency, actors, and transfer between domains.
https://docs.swift.org/swift-book/LanguageGuide/Concurrency.html

F04 Rust. Platform Support. Compilation targets and compiler support levels.
https://doc.rust-lang.org/rustc/platform-support.html

F05 Android. Android ABIs. Distinction between instructions, ABI, and binary compatibility.
https://developer.android.com/ndk/guides/abis

F06 Microsoft. Add Arm support to your Windows app. Native execution, emulation, and ARM64 dependencies.
https://learn.microsoft.com/en-us/windows/arm/add-arm-support

F07 Apple. Finish tasks in the background, WWDC25. Task continuation and conditional background GPU access.
https://developer.apple.com/videos/play/wwdc2025/227/

F08 Android. Background tasks overview. Work models and background restrictions.
https://developer.android.com/develop/background-work/background-tasks

F09 Apple MLX. Official repository. APIs, execution devices, and installation variants.
https://github.com/ml-explore/mlx

F10 Apple. Optimize your Core ML usage, WWDC22. CPU, GPU, and Neural Engine use according to the model.
https://developer.apple.com/videos/play/wwdc2022/10027/

F11 NVIDIA. CUDA Installation Guide for Linux. Qualified architectures and environments.
https://docs.nvidia.com/cuda/cuda-installation-guide-linux/

F12 AMD. ROCm compatibility matrix. Relationships between hardware, operating system, firmware, drivers, and software.
https://rocm.docs.amd.com/en/latest/compatibility/compatibility-matrix.html

F13 Intel Gaudi. Intel Gaudi Software Suite. Execution components and HCCL communication.
https://docs.habana.ai/en/latest/Gaudi_Overview/Intel_Gaudi_Software_Suite.html

F14 Intel OpenVINO. System Requirements. CPU, GPU, and NPU profiles.
https://docs.openvino.ai/2026/about-openvino/release-notes-openvino/system-requirements.html

F15 Kubernetes. Schedule GPUs. GPU resources and node selection.
https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/

F16 Slurm. Generic Resource Scheduling. Allocation of resources such as GPUs.
https://slurm.schedmd.com/gres.html

F17 ONNX Runtime. QNN Execution Provider. Integration with Qualcomm accelerators and specific requirements.
https://onnxruntime.ai/docs/execution-providers/QNN-ExecutionProvider.html

F18 JAX. Installation. Packages and requirements by platform and accelerator, including Cloud TPU.
https://docs.jax.dev/en/latest/installation.html

F19 AWS. What is AWS Neuron. Compilation and execution on Trainium and Inferentia.
https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/what-is-neuron.html

F20 NVIDIA. NVIDIA Collective Communications Library. Communication between GPUs and nodes in the NVIDIA ecosystem.
https://developer.nvidia.com/nccl

F21 AMD. RCCL documentation. Distributed communication optimized for AMD GPUs.
https://rocm.docs.amd.com/projects/rccl/en/latest/

F22 OpenXLA. PJRT Uniform Device API. Device interface through plugins.
https://openxla.org/xla/pjrt

F23 Python Packaging. Platform compatibility tags. Compatibility across Python, ABI, and platform.
https://packaging.python.org/en/latest/specifications/platform-compatibility-tags/

F24 Docker. Multi-platform builds. Image variants by operating system and architecture.
https://docs.docker.com/build/building/multi-platform/

F25 Model Context Protocol. Specification, revision 2026-07-28. Protocol roles, capability rules and revision-specific integration contracts.
https://modelcontextprotocol.io/specification/2026-07-28

F26 Model Context Protocol. Security Best Practices, revision 2026-07-28. Trust boundaries, token handling, endpoint validation and local transport risks.
https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

F27 JSON Schema. Draft 2020-12. Structural validation vocabulary used by the companion schemas.
https://json-schema.org/draft/2020-12

F28 OpenTelemetry. GenAI Semantic Conventions repository. Versioned mappings for traces, metrics and events; TriX must retain her own evidence contracts.
https://github.com/open-telemetry/semantic-conventions-genai

F29 SLSA. Specification version 1.2. Supply-chain provenance, artifacts and verification claims.
https://slsa.dev/spec/v1.2/

F30 W3C. Web Content Accessibility Guidelines 2.2. Testable accessibility criteria for web content.
https://www.w3.org/TR/WCAG22/